

04. Cache Latency & Memory Performance

1. Compiler Optimization Can Break a Benchmark

Consider:

```
for (...) {  
    p = nodes[p].next;  
}
```

If the final value of `p` is never used, the compiler may determine that the loop has **no observable effect**.

What can happen?

```
Code  
↓  
Compiler sees final p is unused  
↓  
Loop has no observable result  
↓  
Compiler may eliminate the loop  
↓  
Benchmark measures almost nothing  
↓  
Unrealistically tiny / near-zero latency
```

Solution: Make the result observable

For example:

```
return {  
    elapsed_ns / static_cast<double>(measure_steps),  
    p  
};
```

Then later:

```
auto result = benchmark();  
int final_index = result.final_index;
```

Now the compiler must preserve the computation because the result is actually consumed.

Interview keyword

Observable result

A benchmark must ensure the work being measured cannot simply be optimized away.

2. Cache Lines

CPUs generally transfer/cache memory in **cache-line-sized blocks**, rather than loading one individual variable at a time.

A common cache-line size is:

64 bytes

Suppose:

`int` = 4 bytes

Then approximately:

$64 / 4 = 16$ ints

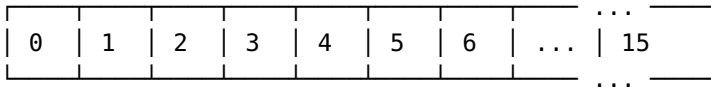
can fit in one cache line.

Example:

```
std::vector<int> data(1000);  
  
data[0];  
data[1];  
data[2];  
data[3];
```

Conceptually:

Cache line (64 bytes)



If the cache line containing `data[0]` is loaded, nearby elements such as:

```
data[1]
data[2]
data[3]
```

may already be present in cache.

This is called:

Spatial Locality

Definition:

If you access one memory location, nearby memory locations are likely to be accessed soon.

Important distinction

Say:

"They are on the same cache line."

Not:

"They are on the same cache."

The problem operates at the **cache-line level**.

3. Spatial Locality

Sequential memory access benefits from spatial locality.

Example:

```
for (int i = 0; i < N; ++i) {  
    sum += data[i];  
}
```

Memory accesses are:

```
data[0]  
data[1]  
data[2]  
data[3]  
...
```

Because these values are close together in memory, cache lines brought into the CPU tend to contain multiple useful elements.

Good locality

```
0 → 1 → 2 → 3 → 4 → 5
```

Poor locality

```
0 → 900 → 37 → 812 → 12
```

The second pattern can cause more cache misses.

4. False Sharing

Example:

```
struct Data {  
    uint64_t a; // Thread 1  
    uint64_t b; // Thread 2  
};
```

Suppose `a` and `b` are on the **same cache line**.

Thread 1 repeatedly modifies:

```
a++;
```

Thread 2 repeatedly modifies:

```
b++;
```

Even though `a` and `b` are **different variables**, they share the same cache line.

This creates **false sharing**.

Complete chain

```
a and b are different variables
      ↓
a and b are on the same cache line
      ↓
False sharing
      ↓
Unnecessary cache-coherence traffic
      ↓
Cache-line bouncing between cores
      ↓
Performance degradation
```

Why?

Cache coherence operates at the **cache-line granularity**.

The CPU does not normally say:

"Only `a` changed."

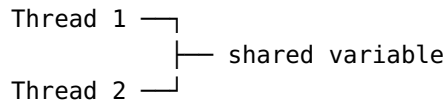
It has to maintain coherence for the cache line containing `a` and `b`.

So when different cores repeatedly modify different variables on that same line, ownership of the line can move between cores.

5. False Sharing vs Real Sharing

Real sharing

Two threads intentionally access the same data:



The communication is intentional.

False sharing

Threads use different variables:

```
Thread 1 → a
Thread 2 → b

a ≠ b
but
a and b → same cache line
```

The coherence traffic is **unnecessary**.

That is why it is called **false** sharing.

6. Cache-Line Bouncing

When different CPU cores repeatedly modify data belonging to the same cache line, the cache line may repeatedly move between cores.

Conceptually:

```
Core 1 owns line
  ↓
Core 2 needs to write
  ↓
Cache line ownership/coherence update
  ↓
Core 2 writes
```

↓
Core 1 needs to write
↓
Cache line moves again

Repeated movement = **cache-line bouncing**

This can create significant latency in multithreaded systems.

7. HFT Importance

These concepts matter heavily in low-latency C++ systems.

Performance can be affected by:

Memory layout
↓
Cache-line usage
↓
Cache hits / misses
↓
Cache coherence
↓
Core-to-core communication
↓
Latency

In HFT, even small sources of unnecessary memory traffic can matter because systems may process very large numbers of events with strict latency requirements.

8. Key C++ Example: Avoiding False Sharing

Instead of:

```
struct Counters {  
    uint64_t thread1_count;  
    uint64_t thread2_count;  
};
```

you can separate frequently modified data so different threads do not contend on the same cache line.

Conceptually:

```
struct alignas(64) Counter1 {  
    uint64_t value;  
};  
  
struct alignas(64) Counter2 {  
    uint64_t value;  
};
```

The goal is to prevent independently modified variables from unnecessarily sharing a cache line.

9. Questions We Solved

Q1. Why can a benchmark report extremely tiny latency?

Answer:

Because the compiler may recognize that the computation produces no observable result and optimize the loop away.

Q2. Why can `data[1]`, `data[2]`, and `data[3]` be cheap after `data[0]`?

Answer:

Because nearby integers can reside in the same cache line.

For a 64-byte cache line and 4-byte `int`:

```
64 / 4 = 16 ints
```

This is **spatial locality**.

Q3. Complete false-sharing chain

```
Different variables  
    ↓  
Same cache line  
    ↓  
False sharing  
    ↓  
Unnecessary cache-coherence traffic
```

↓
Cache-line bouncing
↓
Performance degradation

10. Interview One-Liners

Cache line

A cache line is a fixed-size block of memory transferred between cache levels as a unit.

Spatial locality

Accessing one memory location often makes nearby locations likely to be accessed soon.

False sharing

False sharing occurs when independent variables modified by different threads occupy the same cache line, causing unnecessary coherence traffic.

Cache-line bouncing

Cache-line bouncing is repeated movement or ownership transfer of a cache line between CPU cores due to conflicting accesses.

Observable result

A benchmark must make its result observable so the compiler cannot legally eliminate the measured work.

11. Memory Hooks

Remember these four:

BENCHMARK
→ Make result observable

CACHE
→ Think in cache lines

ARRAY

→ Nearby data benefits from spatial locality

MULTITHREADING

→ Same cache line + different writers = false sharing

10-second recall

64-byte cache line

↓

many nearby values

↓

spatial locality

Different variables

↓

same cache line

↓

false sharing

↓

coherence traffic

↓

cache-line bouncing

↓

slower program

Benchmark result unused

↓

compiler may remove work

↓

bad benchmark